Vulnerability Scanning Techniques

1. Introduction

Vulnerability scanning is a critical phase in cybersecurity used to identify, analyze, and prioritize security weaknesses in systems, networks, and applications. It helps organizations proactively detect vulnerabilities before attackers exploit them.

---

2. Core Concepts of Vulnerability Scanning

2.1 Types of Scanning

1. Network Scanning

- Focuses on identifying open ports, services, and network-level vulnerabilities.
- Common tool: Nmap
- Example:
    - Detect open ports (e.g., 22 for SSH, 80 for HTTP)
    - Identify running services and OS versions

---

2. Application Scanning

- Targets vulnerabilities in web applications such as:
    - SQL Injection
    - Cross-Site Scripting (XSS)
- Common tool: Nikto
- Example:
    - Detect outdated server software
    - Identify insecure configurations

---

3. Authenticated vs Unauthenticated Scans

Scan Type	Description	Advantage
Authenticated Scan	Uses valid credentials	Deep and accurate scanning
Unauthenticated Scan	No login access	Simulates attacker perspective

2.2 Vulnerability Scoring (CVSS)

- CVSS = Common Vulnerability Scoring System
- Used to measure severity of vulnerabilities on a scale from 0 to 10

Example:

- CVE-2017-5638
  - Severity: Critical (CVSS ~10)
  - Type: Remote Code Execution (RCE)

CVSS Score Interpretation:

Score Range	Severity
0.0 – 3.9	Low
4.0 – 6.9	Medium
7.0 – 8.9	High
9.0 – 10	Critical

2.3 False Positives

- A false positive occurs when a scanner reports a vulnerability that does not actually exist.

Example:

- Scanner reports port 80 open → but service is filtered or secured.

Validation Methods:

- Manual verification (e.g., using telnet, curl)
- Cross-check with multiple tools
- Service/version confirmation

3. Key Objectives of Vulnerability Scanning

- Identify security weaknesses
- Prioritize risks using CVSS scoring
- Reduce attack surface
- Ensure compliance with security standards
- Improve overall system security posture

---

4. Learning Resources & Standards

4.1 OWASP Testing Guide

- Focuses on web application vulnerabilities
- Covers:
  - SQL Injection
  - Authentication flaws
  - Session management issues

4.2 NIST SP 800-115

- Provides structured methodology for:
  - Vulnerability scanning
  - Penetration testing
  - Security assessment

4.3 Case Study: WannaCry ransomware attack

- Exploited SMB vulnerability (EternalBlue)
- Demonstrates:

  - Importance of patching
  - Role of CVSS in prioritization
- High CVSS score → Critical risk → Immediate patch required

5. Practical Understanding

To strengthen your understanding:

Practice Activities:

- Perform network scans using Nmap
- Scan a test web app using Nikto
- Compare authenticated vs unauthenticated scans
- Analyze scan results and assign CVSS scores

---

6. Conclusion

Vulnerability scanning is an essential cybersecurity practice that enables organizations to detect and mitigate risks proactively. By understanding scanning techniques, CVSS scoring, and validation methods, students can develop strong foundational skills in security assessment and risk management.

Penetration Testing Techniques

1. Introduction

Penetration testing (pentesting) is a controlled and authorized simulation of cyberattacks on a system, network, or application to identify and exploit vulnerabilities. It helps organizations evaluate their security posture and improve defenses.

---

2. Core Concepts of Penetration Testing

2.1 Phases of Penetration Testing

Penetration testing follows a structured lifecycle to ensure thorough and systematic assessment.

1. Reconnaissance (Information Gathering)

- Collect information about the target without direct interaction (passive) or with interaction (active).
- Goal: Understand target infrastructure.

Tools:

- Shodan (OSINT)

Activities:

- Domain and IP identification
- Employee/email harvesting
- Technology stack detection

---

2. Scanning

- Identify open ports, services, and vulnerabilities.

Tools:

- Nessus

Activities:

- Port scanning
- Service enumeration
- Vulnerability detection

---

3. Exploitation

- Attempt to exploit identified vulnerabilities to gain access.

Tools:

- Metasploit Framework

Activities:

- Remote code execution
- Credential attacks
- Exploiting misconfigurations

---

4. Post-Exploitation

- Maintain access and assess impact.

Activities:

- Privilege escalation
- Data exfiltration
- Lateral movement

---

5. Reporting

- Document findings, risks, and remediation steps.

Includes:

- Vulnerability description
- CVSS score
- Proof of concept (PoC)
- Mitigation recommendations

---

3. Penetration Testing Methodologies

3.1 PTES

- A comprehensive framework defining pentesting phases.

Key Phases:

1. Pre-engagement interactions
2. Intelligence gathering
3. Threat modeling

---

4. Vulnerability analysis
5. Exploitation
6. Post-exploitation
7. Reporting

Example:

- Define scope for web application testing before starting.

3.2 OWASP WSTG

- Focuses on web application security testing.

Covers:

- Authentication testing
- Session management
- Input validation (e.g., SQL Injection, XSS)

---

4. Ethical Considerations in Penetration Testing

Ethics and Legal Requirements

Penetration testing must always follow strict ethical guidelines:

- Obtain **written authorization** before testing
- Clearly define **scope and boundaries**
- Avoid **data destruction or service disruption**
- Maintain **confidentiality of findings**

Important:

Unauthorized testing is illegal and can lead to serious consequences.

---

5. Key Objectives of Penetration Testing

- Identify exploitable vulnerabilities

- Assess real-world attack impact
- Validate security controls
- Provide actionable remediation steps
- Ensure compliance with security standards

---

6. Learning Resources & Standards

6.1 PTES

- Learn detailed pentesting phases and workflows

6.2 OWASP WSTG

- Learn web application security testing techniques

7. Practical Understanding

Practice Activities:

- Perform reconnaissance using Shodan
- Scan a vulnerable machine using Nessus
- Exploit vulnerabilities using Metasploit
- Practice privilege escalation in a lab environment
- Create a professional pentest report

---

8. Conclusion

Penetration testing is a structured and ethical approach to identifying and exploiting vulnerabilities in systems. By understanding its phases, methodologies, and ethical considerations, students can develop strong practical and analytical cybersecurity skills.

Exploit Development Basics

---

1. Introduction

Exploit development is the process of creating code or techniques that take advantage of vulnerabilities in a system. It is a critical skill in penetration testing and helps security professionals understand how attacks work so they can defend against them.

---

2. Core Concepts of Exploit Development

2.1 Types of Exploits

---

1. Buffer Overflow

- Occurs when a program writes more data to a buffer than it can hold.
- This can overwrite memory and allow execution of malicious code.

Example:

- Overwriting return address to redirect execution flow.

Key Idea:

- Control program execution by manipulating memory.

---

2. SQL Injection (SQLi)

- Occurs when user input is not properly sanitized in database queries.

Example:

*SELECT * FROM users WHERE username = 'admin' AND password = '1234';*

- Attack input:

' OR '1'='1

Impact:

- Authentication bypass
- Data leakage

---

3. Cross-Site Scripting (XSS)

- Occurs when untrusted input is executed in a web browser.

Example:

<script>alert('XSS')</script>

Cause:

- Unescaped user input in web pages

Impact:

- Session hijacking
- Credential theft

---

3. Exploit Writing Basics

What is Exploit Writing?

- Creating scripts or code to trigger a vulnerability.

Example Approach (Python-Based Exploit)

- Often used in buffer overflow attacks.

Steps:

1. Identify vulnerable program
2. Find buffer size
3. Locate return address
4. Inject payload (shellcode)

Learning Resource:

- Exploit Database (Exploit-DB)
    - Provides Proof-of-Concept (PoC) exploits
    - Helps understand real-world vulnerabilities

---

4. Security Mitigations

Modern systems include protections to prevent exploitation:

4.1 ASLR (Address Space Layout Randomization)

- Randomizes memory locations
- Makes it difficult to predict target addresses

4.2 WAF (Web Application Firewall)

- Filters malicious HTTP requests
- Blocks attacks like SQLi and XSS

4.3 Patching

- Updating software to fix vulnerabilities

Example:

- Fixing known CVEs with security updates

---

5. Key Objectives of Exploit Development

- Understand how vulnerabilities are exploited
- Develop proof-of-concept exploits safely
- Test system defenses
- Improve secure coding practices

---

6. Learning Resources

6.1 Exploit Database

- Study real exploit code
- Analyze PoCs for different vulnerabilities

6.2 TCM Security

- Beginner-friendly exploit development tutorials

6.3 TryHackMe

- Practice labs like:
    - Buffer overflow rooms
    - Web exploitation challenges

---

7. Conclusion

Exploit development helps cybersecurity professionals understand how attackers think and operate. By learning exploit types, writing basic exploits, and understanding mitigations, students can build strong offensive and defensive security skills.